AI Image Generator

12. Operations and Troubleshooting

Version 2.0
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Z-Image Turbo, Stable Diffusion XL and its best fine-tunes, FLUX.1, LoRAs and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and a live Admin panel.

12. Operations and Troubleshooting

12.1 Starting and stopping

Action	How
Start	Double-click run-app.bat, or `.\.venv\Scripts\python.exe app.py` in the project folder; or `docker compose up`
Confirm ready	Console shows "SDXL base 1.0 ready in N s", "Upscaler ready", "Running on local URL: http://127.0.0.1:7860"
Stop	Ctrl+C in the console, or close the window; `docker compose down` for the container
Change port or address	Set GRADIO_SERVER_PORT or GRADIO_SERVER_NAME before starting; FRAZILE_NO_BROWSER=1 suppresses the browser tab
Expose on the LAN	GRADIO_SERVER_NAME=0.0.0.0. There is no authentication, so only do this on a trusted network

12.2 Logs

The app prints one line per loaded model, per generated image and per upscale, for example:

```
Loading Juggernaut XL v9 ...
Juggernaut XL v9 ready in 26.3s  (scheduler: DPM++ 2M Karras)
#12 juggernaut saved 20261022-101040_seed1746058488.png  832x1216  8.0s  peak 13.1 GB  56.0 C
#12 upscaled -> 1463x2560 in 3.56s
```

To keep a log file, redirect: `.\.venv\Scripts\python.exe -u app.py > app.log 2>&1`. Inside Docker use `docker compose logs -f`.

12.3 Known issues encountered during development and their fixes

Symptom	Cause	Fix applied
AttributeError: 'StableDiffusionXLPipeline' object has no attribute 'enable_vae_slicing'	diffusers 0.41 removed the pipeline-level helper	Call pipe.vae.enable_slicing() instead
Parser ignored "seed 42" and misread "21:9 steps 45" as 9 steps	Directive removal used offsets from a stale lowercase copy of the prompt	Rewrote removal to search and cut the live prompt string each time
Regex word boundaries silently vanished	A patch script wrote "\b" through a normal Python string, turning it into a backspace	Patches now use raw strings or the file is rewritten whole
compel: 'EmbeddingsProviderMulti' object has no attribute 'empty_z'	compel's pad_conditioning_tensors_to_same_length does not support SDXL's two encoders, so a long prompt with a short negative failed	models.py pads the shorter embedding with chunks of the empty-prompt embedding
Theme build failed: Base.set() got an unexpected keyword argument 'color_accent_dark'	Not every Gradio theme variable has a dark-mode twin	build_theme() filters keys against the signature of Base.set()
Admin refresh took 3 s every time	Walking the 39 GB Hugging Face cache and every process for thread counts	Folder sizes cached 60 s, thread count cached 30 s; refresh is 0.7 s
Segmentation fault while generating with Z-Image Turbo	enable_model_cpu_offload() with the 23 GB fp32 transformer on a 16 GB RAM machine	Transformer loaded shard by shard onto the GPU in bf16 with device_map; text encoder in 8-bit; bf16 copy cached under models/cache/
Z-Image generation took 78 to 128 s	The web app was still holding 7 GB of VRAM during the benchmark, so Windows paged GPU memory	Stop other GPU processes before benchmarking; with the GPU free the same images take 8.6 to 9.8 s

Symptom	Cause	Fix applied
Second Qwen-Image image took 207 s (first took 14 s)	VAE decode pushed the peak to 21.7 GB of 24 and allocator fragmentation kept it there, so Windows paged VRAM	VAE slicing and tiling (peak 18.0 GB), torch.cuda.empty_cache() after every image for the big flow models, and PYTORCH_CUDA_ALLOC_CONF=expandable_segments; every image now takes 13 to 14 s
"strict" reported higher guidance for FLUX	Fidelity words were noted before the model was known	apply_defaults() notes that the model has no guidance scale
Space before a comma after a removed tag ("cat , quick")	Tag removal left the separator behind	Tidy step also collapses whitespace before commas
pip could not write pymupdf.exe in C:\Python312\Scripts	Permission problem in the system Python's Scripts folder	Not a project issue; the venv is unaffected. The documentation tooling used the system Python only for one-off edits
UnicodeEncodeError when printing reports from scripts	Windows console code page cp1252 cannot print the arrow and dot characters	Set PYTHONIOENCODING=utf-8 when printing reports; the browser is unaffected
torchvision installed as the CPU build	pip pulled torchvision from PyPI before the CUDA index was consulted	Reinstalled from the cu126 index; requirements.txt documents the order

12.4 Troubleshooting guide

Problem	Check	Resolution
"CUDA GPU not available" at startup	`python -c "import torch; print(torch.version.cuda)"` prints None	Reinstall torch from the cu126 index inside the virtual environment
Port 7860 already in use	Task Manager shows a python.exe from a previous run	End that process, or set GRADIO_SERVER_PORT
"Unknown model 'x'" in the preview line	Typo, or a checkpoint added without restarting	Use a key from the Models accordion; restart after adding files to models/checkpoints/
Generation waits 30 s before starting	The progress bar says "Loading <model>"	Normal for a model switch. Keep the Model box on one model, or Auto, for consecutive prompts
First use of a model waits minutes	Progress says "downloading ~6.5 GB"	Wait; check the HF cache folder growth
"FLUX.1 [schnell] is gated"	No Hugging Face token stored	Accept the terms on the model page and run huggingface-cli login (section 4.10)
LoRA has no effect and the notes say "not found"	File name does not match the tag	Compare with the LoRA list in the Models accordion; matching ignores case and punctuation
Out of memory during generation	Other GPU programs running; or a card under 12 GB; or Z-Image or FLUX on under 24 GB	Close other GPU apps; use generate.py which offloads to CPU on small cards; Z-Image and FLUX need 24 GB
Z-Image is very slow (over a minute per image)	Another process holds several GB of VRAM; nvidia-smi shows memory near 24 GB	Close it; Z-Image needs about 18 GB free
Telemetry fields show None	nvidia-smi not on PATH	Reinstall the NVIDIA driver, or add C:\Windows\System32 to PATH
Upscaler weights missing	FileNotFoundError naming models/RealESRGAN_x4plus.pth	Download from the Real-ESRGAN v0.1.0 release into models/
Upscale bar does nothing	No generation in this browser session	Generate first; the bar works on the records of the last generation
History tab empty	New database, or generations.db moved	Generate an image; or restore the file next to db.py

Problem	Check	Resolution
Admin panel shows 0 B/s network	First sample	Refresh again; rates need two samples
Browser slow to show an 8K result	30 MB PNG transfer	Expected; open the file from outputs/ directly
docker compose fails with "could not select device driver"	GPU support not enabled in Docker	Install WSL 2 and Docker Desktop (Windows) or nvidia-container-toolkit (Linux)

12.5 Maintenance

- Outputs grow by 1.5 MB per native image and 3 to 31 MB per upscaled image. Prune outputs/ periodically; the History tab will show "not found" for deleted files. The Admin panel shows the folder size.
- Each built-in model is 6.5 GB in the Hugging Face cache. Delete a models--* folder there to reclaim space; it downloads again on next use.
- Back up outputs/, generations.db and models/ together so paths stay valid.
- To reset history, stop the app and delete generations.db; it is recreated empty on next start.
- To update dependencies, upgrade inside the virtual environment and rerun the parser self-test, models.py, sysinfo.py and one generation per family. Pin versions in requirements.txt if a breaking change is found.

12.6 Security notes

- The server listens on localhost only by default and has no authentication. The Docker image binds all interfaces inside the container; the compose file publishes the port on the host, so treat that like a LAN exposure.
- No telemetry or prompts are sent anywhere. Network access is limited to model downloads from Hugging Face and the optional reachability check in the Admin panel, which only opens a TCP connection.
- The Admin panel reveals hostnames, IP addresses, MAC addresses, listening ports and process names. Anyone who can reach the page can read them.
- Prompts are stored verbatim in generations.db. Treat the file with the same care as the images.